Министерство образования и науки РФ

Пермский национальный исследовательский политехнический университет

Электротехнический факультет

Кафедра "Информационные технологии и автоматизированные системы"

Информатика в приложении к отрасли

Лабораторная работа № 2

Тема: "Методы решения нелинейных уравнений"

Вариант 4

Выполнил: студент группы РИС-25-2б Бастраков Данил

Проверил: профессор кафедры ИТАС Шварц К. Г.

Дата: _____

Пермь
2026

Содержание

1 Постановка задачи **2**

2 Отделение корней **2**

3 Краткое описание методов **4**
3.1 Метод половинного деления (дихотомии) 4
3.2 Метод хорд 5
3.3 Метод касательных (Ньютона) 6
3.4 Комбинированный метод хорд и касательных 7
3.5 Метод простых итераций 8

4 Проверка условий сходимости и выбор начального приближения **9**

5 Программная реализация **10**

6 Результаты вычислений **11**
6.1 Сводная таблица и проверка подстановкой 13

7 Обсуждение результатов **14**

A Листинг программы **15**

1. Постановка задачи

Цель работы - отделить корни нелинейного уравнения и уточнить их с заданной точностью методами половинного деления, хорд, касательных (Ньютона), комбинированным методом хорд и касательных, а также методом простых итераций.

Дано нелинейное уравнение (вариант 4):

$$f(x) = x^3 - 0{,}1\,x^2 + 0{,}4\,x - 1{,}5 = 0. \tag{1}$$

Требуется:

1) отделить корни уравнения аналитически;
2) уточнить корень каждым из указанных методов с точностью $\varepsilon = 10^{-6}$;
3) обосновать выбор начального приближения и проверить условия сходимости методов;
4) привести программную реализацию, сводную таблицу результатов и сравнить методы по числу шагов и характеру приближения.

2. Отделение корней

Обозначим $f(x) = x^3 - 0{,}1x^2 + 0{,}4x - 1{,}5$. Найдём первую производную:

$$f'(x) = 3x^2 - 0{,}2x + 0{,}4. \tag{2}$$

Дискриминант этого квадратного трёхчлена равен

$$D = (-0{,}2)^2 - 4 \cdot 3 \cdot 0{,}4 = 0{,}04 - 4{,}8 = -4{,}76 < 0, \tag{3}$$

поэтому $f'(x) > 0$ при всех x. Функция строго возрастает и уравнение (1) имеет *ровно один* действительный корень.

Составим таблицу знаков функции (табл. 1).

Таблица 1 – Знаки функции $f(x)$ на грубой сетке

x	-2	-1	0	1	2
$f(x)$	$-10{,}7$	$-3{,}0$	$-1{,}5$	$-0{,}2$	$6{,}9$
знак	$-$	$-$	$-$	$-$	$+$

Смена знака происходит на отрезке $[1;2]$, значит $\xi \in [1;2]$. Уменьшим отрезок, исследуя его с шагом $0{,}1$:

$$f(1{,}0) = -0{,}2 < 0, \qquad f(1{,}1) = 0{,}15 > 0,$$

откуда $\boxed{\xi \in [1;\ 1{,}1]}$. Дальнейшее уточнение этого отрезка и выполняют численные методы.

Вторая производная

$$f''(x) = 6x - 0{,}2 \tag{4}$$

на отрезке $[1; 1{,}1]$ положительна ($f''(x) > 0$), то есть функция выпукла вниз. Знак второй производной используется ниже при выборе начального приближения для методов хорд и касательных.

График функции приведён на рис. 1, а расположение корня на отрезке $[1; 1{,}1]$ - на рис. 2.

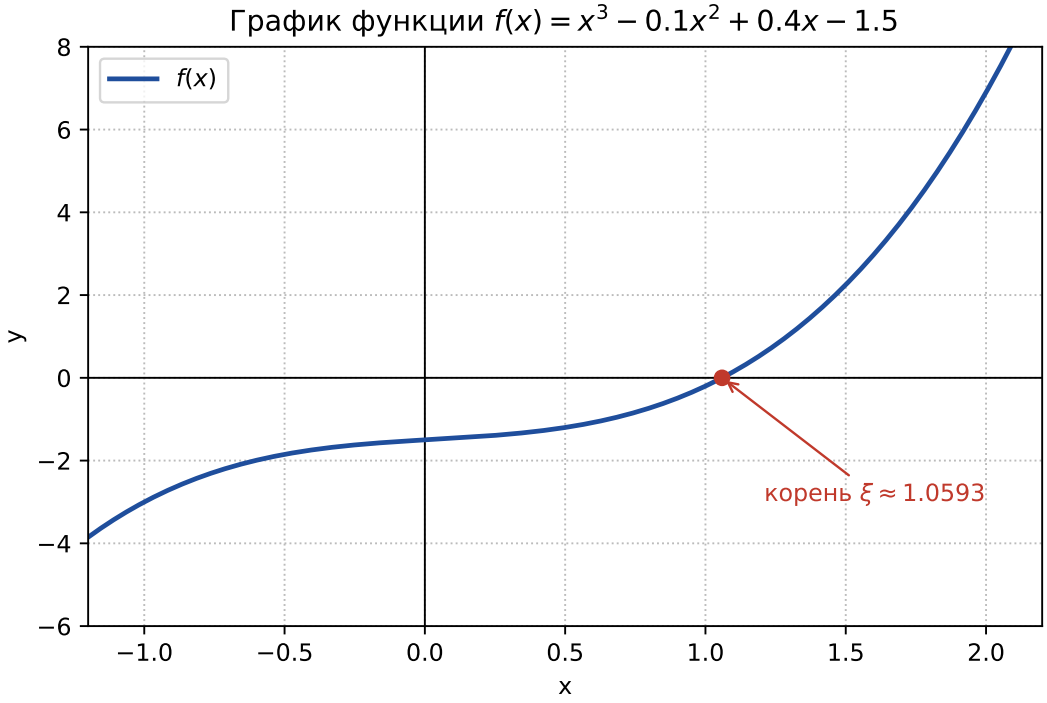


Рис. 1 – График функции $f(x) = x^3 - 0{,}1x^2 + 0{,}4x - 1{,}5$



Рис. 2 – Отделение корня на отрезке $[1; 1{,}1]$

3. Краткое описание методов

Все методы уточнения корня применяются к отрезку $[a; b]$, на концах которого функция принимает значения разных знаков: $f(a) \cdot f(b) < 0$.

3.1. Метод половинного деления (дихотомии)

Отрезок $[a; b]$ делится пополам точкой $c = (a + b)/2$. Из двух половин выбирается та, на концах которой знаки функции различны:

$$\text{если } f(a) \cdot f(c) \leq 0, \text{ то } b := c, \quad \text{иначе } a := c. \tag{5}$$

После n делений длина отрезка равна $(b-a)/2^n$, поэтому метод сходится всегда и со скоростью геометрической прогрессии со знаменателем $1/2$. Условие останова: $b - a < \varepsilon$. Приближение к корню идёт с двух сторон: левые концы a_n не убывают, правые b_n не возрастают (рис. 3).

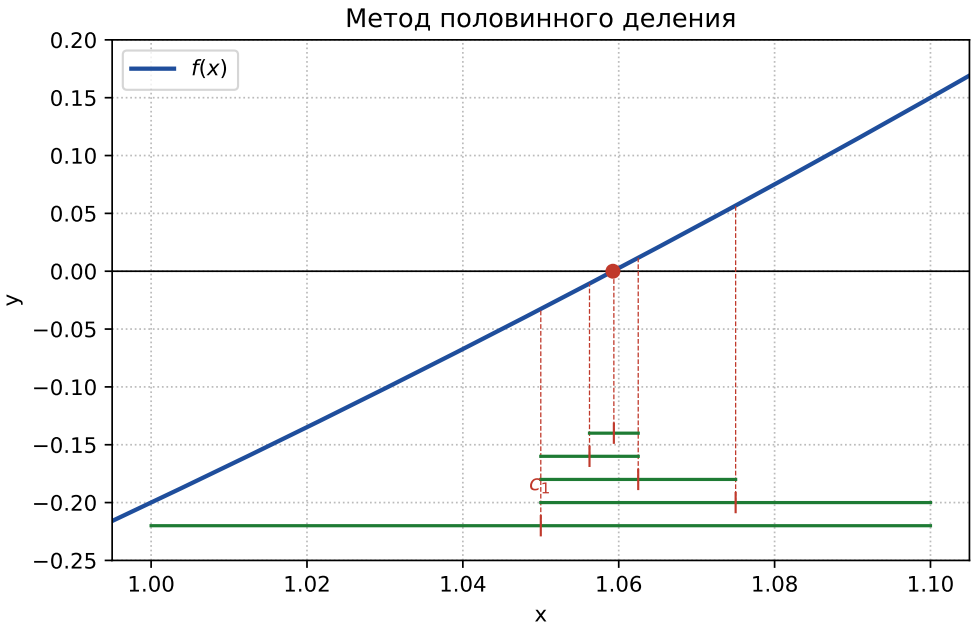


Рис. 3 – Метод половинного деления: последовательное деление отрезка

3.2. Метод хорд

Проводится хорда через точки $(a, f(a))$ и $(b, f(b))$; точка её пересечения с осью Ox принимается за новое приближение. Один конец отрезка неподвижен - это тот, для которого $f(x) \cdot f''(x) > 0$; начальным приближением служит противоположный конец. В нашем случае $f(b) \cdot f''(b) > 0$, поэтому неподвижен конец b, а $x_0 = a$:

$$x_{n+1} = x_n - \frac{f(x_n)}{f(b) - f(x_n)}\,(b - x_n). \tag{6}$$

Последовательность монотонна и приближается к корню с одной стороны (рис. 4). Скорость сходимости - линейная.

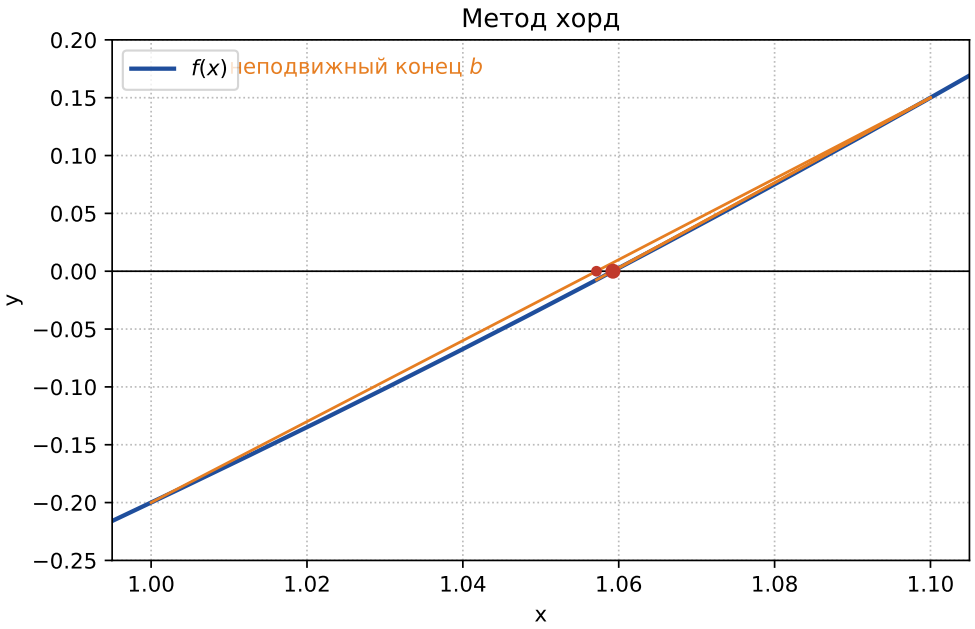


Рис. 4 – Метод хорд: приближение с одной стороны

3.3. Метод касательных (Ньютона)

В точке x_n проводится касательная к графику функции; точка её пересечения с осью Ox даёт следующее приближение:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}. \tag{7}$$

За начальное приближение выбирается тот конец отрезка, где $f(x_0) \cdot f''(x_0) > 0$ (условие сходимости Фурье). В нашем случае $x_0 = b = 1{,}1$. Метод имеет квадратичную сходимость и приближается к корню с одной стороны (рис. 5).

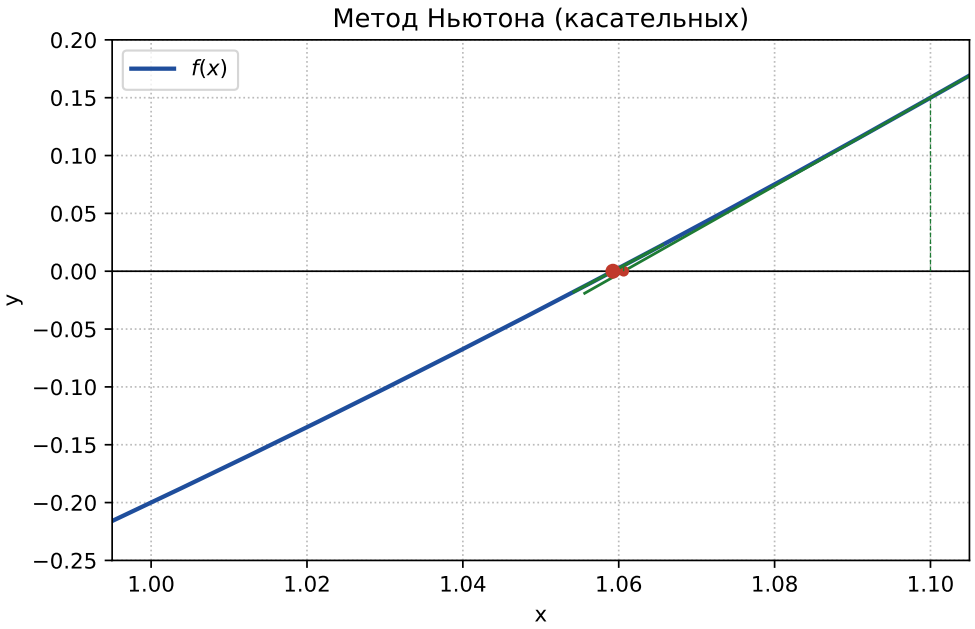


Рис. 5 – Метод касательных: построение касательных

3.4. Комбинированный метод хорд и касательных

Корень уточняется одновременно с двух сторон. Обозначим x_n - приближение по недостатку (методом хорд), y_n - по избытку (методом касательных). Хорды строятся из конца, где $f(x) \cdot f''(x) < 0$, а касательные - из конца, где $f(x) \cdot f''(x) > 0$. Тогда

$$x_{n+1} = x_n - \frac{f(x_n)}{f(x_n) - f(y_n)}\,(x_n - y_n), \qquad y_{n+1} = y_n - \frac{f(y_n)}{f'(y_n)}. \tag{8}$$

Процесс заканчивается при $y_n - x_n < \varepsilon$; корень всегда заключён между x_n и y_n (рис. 6). Метод даёт оценку погрешности на каждом шаге и сходится быстрее метода хорд.

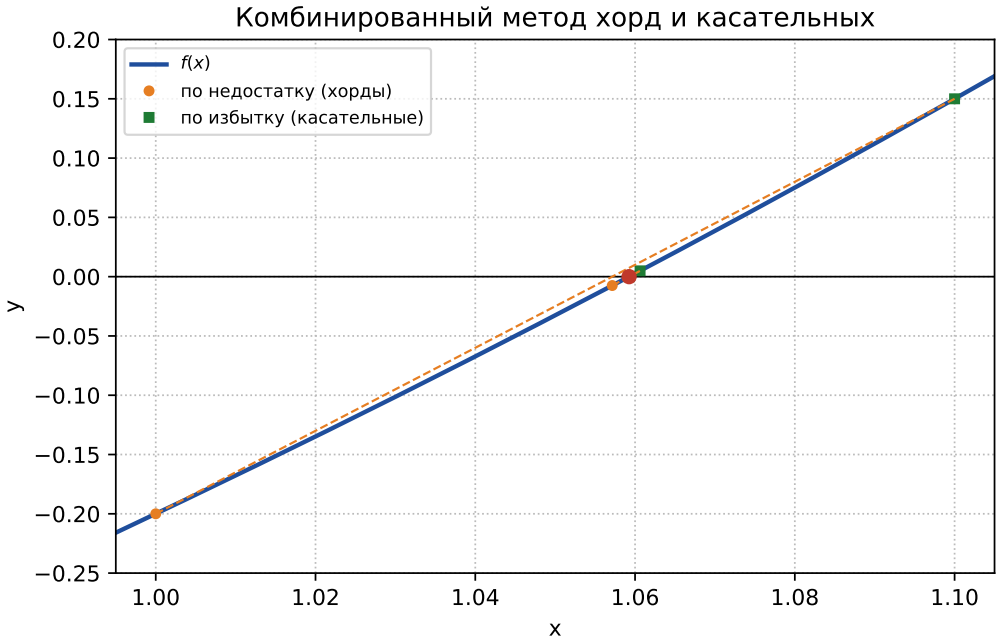


Рис. 6 – Комбинированный метод: хорды слева, касательные справа

3.5. Метод простых итераций

Уравнение $f(x) = 0$ приводится к виду $x = \varphi(x)$. Возьмём

$$\varphi(x) = x - \frac{f(x)}{k}, \tag{9}$$

где параметр k выбирается одного знака с $f'(x)$ и по модулю больше $Q/2$, а $Q = \max_{[a;b]} |f'(x)|$. Тогда

$$\varphi'(x) = 1 - \frac{f'(x)}{k}, \qquad |\varphi'(x)| \le q < 1, \tag{10}$$

что обеспечивает сходимость последовательности $x_{n+1} = \varphi(x_n)$. Оценка погрешности:

$$|\xi - x_n| \le \frac{q}{1-q} |x_n - x_{n-1}|. \tag{11}$$

Приближения подходят к корню с одной стороны (лестница) при $\varphi'(x) > 0$ (рис. 7).

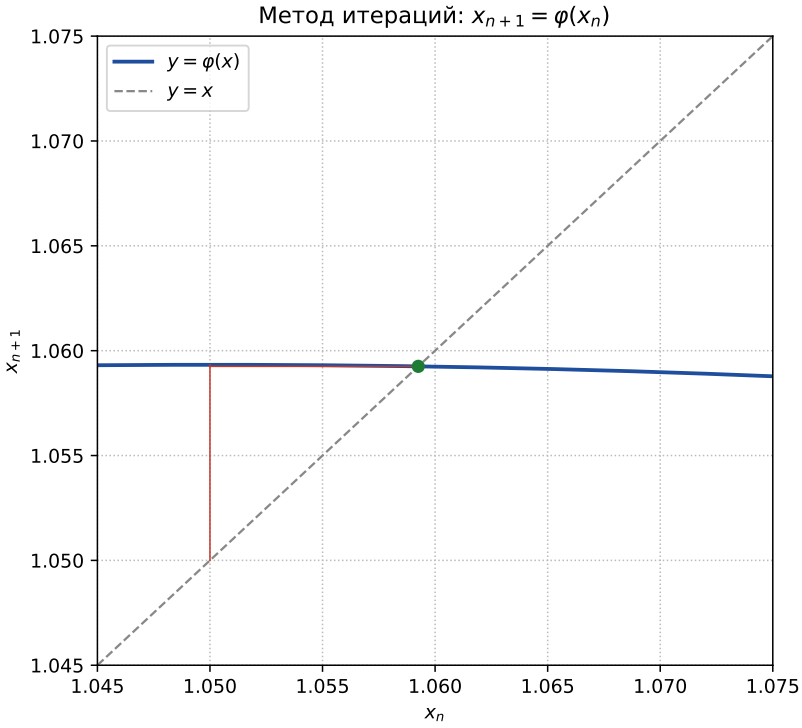


Рис. 7 – Метод итераций: лестница приближений к корню

4. Проверка условий сходимости и выбор начального приближения

На отрезке $[1; 1{,}1]$ имеем $f(1) = -0{,}2 < 0$, $f(1{,}1) = 0{,}15 > 0$, $f''(x) = 6x - 0{,}2 > 0$.

1) **Метод половинного деления.** Так как $f(1) \cdot f(1{,}1) < 0$, метод сходится при любом начальном отрезке; приближение идёт с двух сторон.

2) **Метод хорд.** Вторая производная знакопостоянна на $[1; 1{,}1]$. Неподвижен тот конец, где $f(x)f''(x) > 0$:

$$f(1)f''(1) = (-0{,}2)(5{,}8) < 0, \qquad f(1{,}1)f''(1{,}1) = 0{,}15 \cdot 6{,}4 > 0.$$

Значит неподвижен конец $b = 1{,}1$, начальное приближение $x_0 = a = 1$.

3) **Метод касательных.** Условие Фурье $f(x_0)f''(x_0) > 0$ выполняется при $x_0 = 1{,}1$, поэтому $x_0 = 1{,}1$.

4) **Комбинированный метод.** Хорды - из $x_0 = 1{,}0$ (здесь $ff'' < 0$), касательные - из $y_0 = 1{,}1$ (здесь $ff'' > 0$).

5) **Метод итераций.** $Q = \max|f'| = f'(1{,}1) = 3{,}81$. Выбираем $k = f'(1{,}05) = 3{,}4975$; при этом $|k| > Q/2 = 1{,}905$ и знак k совпадает

со знаком f'. Тогда

$$q = \max_{[1;1,1]} \left| 1 - \frac{f'(x)}{k} \right| \approx 0{,}0893 < 1.$$

Условие сходимости выполнено. Начальное приближение $x_0 = 1{,}05$ (середина отрезка).

5. Программная реализация

Программа написана на языке C# и поддерживает универсальный ввод данных. Если в каталоге программы есть файл `input.txt`, из него читаются коэффициенты уравнения a_3, a_2, a_1, a_0, границы отрезка a, b и шаги сканирования; если файла нет, используются параметры варианта 4. Исходный код разбит на отдельные классы (табл. 2); полный листинг приведён в приложении А.

Таблица 2 – Состав программы

Файл	Назначение
`Equation.cs`	вычисление $f(x)$, $f'(x)$, $f''(x)$; коэффициенты уравнения
`RootSeparation.cs`	отделение корня по таблице знаков
`BisectionSolver.cs`	метод половинного деления
`ChordSolver.cs`	метод хорд
`NewtonSolver.cs`	метод касательных (Ньютона)
`CombinedSolver.cs`	комбинированный метод хорд и касательных
`IterationSolver.cs`	метод простых итераций
`RootResult.cs`	результат работы метода (для сводной таблицы)
`NumberFormat.cs`	форматирование чисел при выводе
`Program.cs`	ввод данных, запуск методов, сводная таблица

Ниже приведён фрагмент реализации - вычисление производных и тело метода касательных.

```
// Значение функции f(x) по схеме Горнера
public static double Value(double x)
{
    return ((c3 * x + c2) * x + c1) * x + c0;
}
```

```

// Первая производная f'(x)
public static double Derivative(double x)
{
    return (3 * c3 * x + 2 * c2) * x + c1;
}

// Вторая производная f''(x)
public static double SecondDerivative(double x)
{
    return 6 * c3 * x + 2 * c2;
}
```

Listing 1 – Производные функции (Equation.cs)

```
do
{
    var fx = Equation.Value(x);
    next = x - fx / Equation.Derivative(x);   // x(n+1) = x(n) - f/f'
    delta = Math.Abs(next - x);
    steps++;
    // вывод строки пошаговой таблицы ...
    x = next;
} while (delta >= epsilon && Math.Abs(Equation.Value(x)) >= epsilon);
```

Listing 2 – Итерационный процесс метода Ньютона (NewtonSolver.cs)

6. Результаты вычислений

Пошаговые расчёты всех методов с точностью $\varepsilon = 10^{-6}$ приведены в табл. 3--7.

Таблица 3 – Метод половинного деления

n	a	b	$c = (a+b)/2$	$b-a$
1	1.00000000	1.10000000	1.05000000	0.10000000
2	1.05000000	1.10000000	1.07500000	0.05000000
3	1.05000000	1.07500000	1.06250000	0.02500000
4	1.05000000	1.06250000	1.05625000	0.01250000
5	1.05625000	1.06250000	1.05937500	0.00625000
6	1.05625000	1.05937500	1.05781250	0.00312500
7	1.05781250	1.05937500	1.05859375	0.00156250
8	1.05859375	1.05937500	1.05898438	0.00078125
9	1.05898438	1.05937500	1.05917969	0.00039062
10	1.05917969	1.05937500	1.05927734	0.00019531
11	1.05917969	1.05927734	1.05922852	0.00009766
12	1.05922852	1.05927734	1.05925293	0.00004883
13	1.05925293	1.05927734	1.05926514	0.00002441
14	1.05925293	1.05926514	1.05925903	0.00001221
15	1.05925293	1.05925903	1.05925598	0.00000610
16	1.05925293	1.05925598	1.05925446	0.00000305
17	1.05925293	1.05925446	1.05925369	0.00000153

Корень: $x = 1{,}05925331$, $f(x) = 4{,}0 \cdot 10^{-7}$, число шагов $n = 17$.

Таблица 4 – Метод хорд

n	x_n	$f(x_n)$	x_{n+1}	$\lvert x_{n+1} - x_n \rvert$
1	1.00000000	-0.20000000	1.05714286	0.05714286
2	1.05714286	-0.00748688	1.05918027	0.00203742
3	1.05918027	-0.00025917	1.05925068	0.00007041
4	1.05925068	-0.00000895	1.05925311	0.00000243

Корень: $x = 1{,}05925311$ ($n = 4$).

Таблица 5 – Метод касательных (Ньютона)

n	x_n	$f(x_n)$	x_{n+1}	$\lvert x_{n+1} - x_n \rvert$
1	1.10000000	0.15000000	1.06062992	0.03937008
2	1.06062992	0.00489899	1.05925484	0.00137508
3	1.05925484	0.00000582	1.05925320	0.00000164

Корень: $x = 1{,}05925320$ ($n = 3$).

Таблица 6 – Комбинированный метод хорд и касательных

n	x_n (недост.)	y_n (изб.)	$f(x_n)$	$f(y_n)$	$y_n - x_n$
1	1.00000000	1.10000000	-0.20000000	0.15000000	0.10000000
2	1.05714286	1.06062992	-0.00748688	0.00489899	0.00348706
3	1.05925068	1.05925484	-0.00000895	0.00000582	0.00000416

Корень: $x = 1{,}05925320$ ($n = 3$).

Таблица 7 – Метод простых итераций ($k = 3{,}4975$)

n	x_n	$f(x_n)$	$\lvert x_n - x_{n-1} \rvert$
1	1.05000000	-0.03262500	0.00932809
2	1.05932809	0.00026620	0.00007611
3	1.05925198	-0.00000433	0.00000124
4	1.05925322	0.00000007	0.00000002

Корень: $x = 1{,}05925320$ ($n = 4$).

6.1. Сводная таблица и проверка подстановкой

Сводные результаты всех методов приведены в табл. 8.

Таблица 8 – Сравнение методов при $\varepsilon = 10^{-6}$

Метод	Задание a, b, x_0	Корень x	Шагов n
половинного деления	$a = 1{,}0$, $b = 1{,}1$	1.05925331	17
хорд	$x_0 = 1{,}0$ (неподв. b)	1.05925311	4
касательных (Ньютона)	$x_0 = 1{,}1$	1.05925320	3
комбинированный	$x_0 = 1{,}0$, $y_0 = 1{,}1$	1.05925320	3
итераций	$a = 1{,}0$, $b = 1{,}1$, $x_0 = 1{,}05$	1.05925320	4

Проверка подстановкой (значения $f(x)$ в найденных корнях):

Метод	$f(x)$
половинного деления	$4{,}0 \cdot 10^{-7}$
хорд	$-3{,}1 \cdot 10^{-7}$
касательных (Ньютона)	$8{,}0 \cdot 10^{-12}$
комбинированный	$-2{,}0 \cdot 10^{-12}$
итераций	$-1{,}1 \cdot 10^{-9}$

7. Обсуждение результатов

Все пять методов дали один и тот же корень $\xi = 1{,}059253$ с точностью не хуже 10^{-6}; при подстановке в уравнение значения $f(x)$ не превышают $4 \cdot 10^{-7}$, что подтверждает правильность расчётов.

Различие методов состоит в скорости и характере сходимости:

- **Метод половинного деления** наиболее надёжен: сходится всегда при $f(a)f(b) < 0$, но медленнее всех - потребовалось 17 шагов; из-за деления пополам число шагов практически не зависит от вида функции, а длина отрезка убывает как 2^{-n}. Приближение идёт с двух сторон.

- **Метод хорд** линеен и монотонен: приближения подходят к корню с одной стороны (по недостатку), корень всё время заключён между x_n и неподвижным концом b. Потребовалось 4 шага; на каждом шаге погрешность убывает примерно в одно и то же число раз.

- **Метод касательных** сходится квадратично: число верных цифр удваивается на каждом шаге, поэтому хватило 3 шагов. Это самый быстрый из методов, но требует вычисления производной и удачного выбора x_0 (условие Фурье). Если x_0 выбрать неудачно, сходимость может нарушиться.

- **Комбинированный метод** дал корень за 3 шага и, в отличие от метода касательных, на каждом шаге даёт *двустороннюю* оценку: корень всегда заключён между x_n и y_n. Это позволяет контролировать погрешность без априорной оценки; при этом метод устойчивее «чистого» метода Ньютона.

- **Метод простых итераций** сошёлся за 4 шага при $q \approx 0{,}09$; скорость сходимости линейная со знаменателем q. Метод прост в реализации, но требует подбора параметра k (условие $|k| > Q/2$ и совпадение знака с f').

Вывод. Для данного уравнения наиболее эффективны комбинированный метод и метод касательных (по 3 шага), однако метод половинного деления и метод итераций проще в реализации и не требуют вычисления производной. Комбинированный метод сочетает высокую скорость метода Ньютона с гарантированной двусторонней оценкой погрешности и потому является предпочтительным при ручных расчётах.

Список литературы

[1] Демидович Б. П., Марон И. А. Основы вычислительной математики. - М.: Наука, 1970.

[2] Бахвалов Н. С., Жидков Н. П., Кобельков Г. М. Численные методы. - М.: БИНОМ. Лаборатория знаний, 2011.

[3] Волков Е. А. Численные методы. - СПб.: Лань, 2008.

A. Листинг программы

Program.cs

```
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;

// Лабораторная работа №2. Методы решения нелинейных уравнений.
// Вариант 4: f(x) = x^3 − 0.1 x^2 + 0.4 x − 1.5 = 0.
// 1) корень отделяется аналитически;
// 2)–6) уточняется методами половинного деления, хорд, касательных,
// комбинированным методом хорд и касательных и методом итераций
// с точностью eps = 10^−6.
//
// Данные можно задать в файле input.txt см(. ReadInput), иначе берётся
// уравнение варианта 4 на отрезке [1; 1.1].
class Program
```

```
{
    private const double Epsilon = 1e-6;

    private static void Main()
    {
        Console.WriteLine("Лабораторная работа №2. Методы решения нелинейных уравнений");
        Console.WriteLine("Вариант 4. Уравнение: f(x) = " + Equation.Formula);
        Console.WriteLine();

        var (a, b, coarseStep, fineStep) = ReadInput();

        var (rootA, rootB) = RootSeparation.Separate(a, b, coarseStep, fineStep);
        Console.WriteLine("Корень отделён на отрезке [" + NumberFormat.Format(rootA, 2)
            + "; " + NumberFormat.Format(rootB, 2) + "].");
        Console.WriteLine();

        var results = new List<RootResult>
        {
            BisectionSolver.Solve(rootA, rootB, Epsilon),
            ChordSolver.Solve(rootA, rootB, Epsilon),
            NewtonSolver.Solve(rootA, rootB, Epsilon),
            CombinedSolver.Solve(rootA, rootB, Epsilon),
            IterationSolver.Solve(rootA, rootB, Epsilon),
        };

        PrintSummary(results);
    }

    // Универсальный ввод данных. Файл input.txt необязательный() содержит:
    //   строка 1: коэффициенты a3 a2 a1 a0;
    //   строка 2: границы отрезка поиска a b;
    //   строка 3: шаги грубого и точного сканирования необязательно().
    // Если файла нет или он не разобран, используются параметры варианта 4.
    private static (double A, double B, double Coarse, double Fine) ReadInput()
    {
        const double defaultCoarse = 1;
        const double defaultFine = 0.1;
        const string fileName = "input.txt";

        if (!File.Exists(fileName))
        {
            Console.WriteLine("Файл input.txt не найден, используется уравнение варианта 4.");
            Console.WriteLine();
            return (-2, 3, defaultCoarse, defaultFine);
        }

        var parts = new List<double>();
        foreach (var line in File.ReadAllLines(fileName))
        {
            var tokens = line.Split((char[]?)null, StringSplitOptions.RemoveEmptyEntries);
            foreach (var token in tokens)
            {
```

```
                if (!double.TryParse(token.Replace(',', '.'), NumberStyles.Float,
                        CultureInfo.InvariantCulture, out var value))
                {
                    Console.WriteLine("input.txt: \"" + token + "\" не является числом, "
                        + "используются параметры варианта 4.");
                    Console.WriteLine();
                    return (-2, 3, defaultCoarse, defaultFine);
                }
                parts.Add(value);
            }
        }

        if (parts.Count < 6)
        {
            Console.WriteLine("input.txt: нужно минимум 6 чисел (a3 a2 a1 a0 a b), "
                + "используются параметры варианта 4.");
            Console.WriteLine();
            return (-2, 3, defaultCoarse, defaultFine);
        }

        Equation.Set(parts[0], parts[1], parts[2], parts[3]);
        var coarse = parts.Count > 6 ? parts[6] : defaultCoarse;
        var fine = parts.Count > 7 ? parts[7] : defaultFine;
        Console.WriteLine("Данные прочитаны из input.txt: f(x) = " + Equation.Formula);
        Console.WriteLine();
        return (parts[4], parts[5], coarse, fine);
    }

    // Сводная таблица: метод, задание отрезка и начального приближения,
    // найденный корень, количество шагов, проверка подстановкой.
    private static void PrintSummary(List<RootResult> results)
    {
        Console.WriteLine("СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ (eps = 10^-6)");
        Console.WriteLine(string.Format("{0,-28}{1,-26}{2,18}{3,8}",
            "Метод", "a, b, x0", "x", "Шагов"));
        foreach (var result in results)
        {
            Console.WriteLine(string.Format("{0,-28}{1,-26}{2,18}{3,8}",
                result.Method, result.Bounds, NumberFormat.Format(result.Root, 8),
                result.Steps));
        }

        Console.WriteLine();
        Console.WriteLine("Проверка подстановкой: значения f(x) в найденных корнях:");
        foreach (var result in results)
        {
            Console.WriteLine("    " + result.Method.PadRight(28) + " f(x) = "
                + NumberFormat.Format(result.Residual, 12));
        }
    }
}
```

Equation.cs

```
using System;
using System.Globalization;
using System.Text;

// Уравнение вида  a3*x^3 + a2*x^2 + a1*x + a0 = 0.
// По умолчанию используется уравнение варианта 4:
//     x^3 − 0.1 x^2 + 0.4 x − 1.5 = 0.
//
// Аналитическое отделение корней. Первая производная
//     f'(x) = 3*a3*x^2 + 2*a2*x + a1
// для варианта 4 имеет дискриминант D = 0.2^2 − 4*3*0.4 = −4.76 < 0, поэтому
// f'(x) > 0 при всех x: функция строго возрастает и уравнение имеет ровно
// один действительный корень. Вторая производная выпукла вниз при x > 0.
static class Equation
{
    private static double c3 = 1.0;
    private static double c2 = −0.1;
    private static double c1 = 0.4;
    private static double c0 = −1.5;

    public static void Set(double a3, double a2, double a1, double a0)
    {
        c3 = a3;
        c2 = a2;
        c1 = a1;
        c0 = a0;
    }

    // Значение функции f(x) по схеме Горнера
    public static double Value(double x)
    {
        return ((c3 * x + c2) * x + c1) * x + c0;
    }

    // Первая производная f'(x)
    public static double Derivative(double x)
    {
        return (3 * c3 * x + 2 * c2) * x + c1;
    }

    // Вторая производная f''(x)
    public static double SecondDerivative(double x)
    {
        return 6 * c3 * x + 2 * c2;
    }

    // Коэффициенты уравнения
    public static double[] Coefficients => new[] { c3, c2, c1, c0 };

    // Текстовая запись уравнения
    public static string Formula
    {
        get
        {
            var text = new StringBuilder();
            text.Append(Term(c3, "x^3", true));
            text.Append(Term(c2, "x^2", false));
            text.Append(Term(c1, "x", false));
```

```
            text.Append(Term(c0, "", false));
            return text.Append(" = 0").ToString();
        }
    }

    // Печатает одно слагаемое. Первое слагаемое печатается без знака '+',
    // единичные коэффициенты (1 и -1) при неизвестном опускаются.
    private static string Term(double value, string suffix, bool first)
    {
        var text = new StringBuilder();
        if (Math.Abs(value) < 1e-15)
        {
            return first ? "0" : string.Empty;
        }

        if (first)
        {
            if (value < 0)
            {
                text.Append('-');
            }
        }
        else
        {
            text.Append(value < 0 ? " - " : " + ");
        }

        var magnitude = Math.Abs(value);
        if (suffix.Length == 0 || Math.Abs(magnitude - 1) > 1e-15)
        {
            text.Append(Format(magnitude));
            if (suffix.Length > 0)
            {
                text.Append(' ');
            }
        }
        text.Append(suffix);
        return text.ToString();
    }

    private static string Format(double value)
    {
        return value.ToString("G", CultureInfo.InvariantCulture);
    }
}
```

RootSeparation.cs

```
using System;

// Отделение корней уравнения. Сначала корни отделяются аналитически
// по( знаку производной устанавливается число корней), затем отрезок
// сужается по таблице знаков функции f(x) до длины не больше заданной.
static class RootSeparation
{
    // Печатает исследование функции и возвращает отрезок [a, b],
    // содержащий единственный корень. Сначала выполняется грубый проход
    // с шагом coarseStep, затем найденный отрезок сужается с шагом fineStep.
```

```
    public static (double A, double B) Separate(double from, double to,
        double coarseStep, double fineStep)
    {
        Console.WriteLine("ОТДЕЛЕНИЕ КОРНЕЙ");
        Console.WriteLine("Уравнение: f(x) = " + Equation.Formula);
        Console.WriteLine("f'(x) = 3x^2 − 0.2x + 0.4, D = 0.04 − 4.8 = −4.76
    < 0 => f'(x) > 0,");
        Console.WriteLine("функция строго возрастает, значит корень
    единственный.");
        Console.WriteLine();

        Console.WriteLine("Грубая таблица знаков функции:");
        var (a, b) = FindSignChange(from, to, coarseStep);
        Console.WriteLine();

        Console.WriteLine("Уточнение отрезка с шагом " + NumberFormat.Format(
    fineStep, 1)
            + ":");
        var refined = FindSignChange(a, b, fineStep);
        Console.WriteLine();
        return refined;
    }

    // Ищет отрезок смены знака функции на [from, to] с заданным шагом,
    // печатая таблицу знаков. Возвращает отрезок [a, b], где f(a)*f(b) < 0.
    private static (double A, double B) FindSignChange(double from, double to
    , double step)
    {
        double previousX = from;
        double previousValue = Equation.Value(from);
        Console.WriteLine(Row(previousX, previousValue));

        for (var x = from + step; x <= to + 1e-9; x += step)
        {
            var value = Equation.Value(x);
            Console.WriteLine(Row(x, value));
            if (Math.Sign(value) != Math.Sign(previousValue))
            {
                return (previousX, x);
            }
            previousX = x;
            previousValue = value;
        }

        throw new InvalidOperationException(
            "На отрезке [" + from + ", " + to + "] смена знака не найдена.");
    }

    // Строка таблицы знаков: x и f(x) со знаком
    private static string Row(double x, double value)
    {
        var sign = value > 0 ? "+" : value < 0 ? "−" : "0";
        return "   x = " + NumberFormat.Cell(x, 2)
            + "   f(x) = " + NumberFormat.Cell(value, 4)
            + "   знак: " + sign;
    }
}
```

BisectionSolver.cs

```
using System;

// Метод половинного деления дихотомии(). Отрезок [a, b], на концах которого
// f(a)*f(b) < 0, делится пополам; выбирается та половина, где знаки функции
// на концах различны. Метод сходится всегда и приближается к корню с двух
// сторон: левые концы a_n образуют неубывающую, а правые b_n — невозрастающую
// последовательность. Условие останова: b − a < eps.
static class BisectionSolver
{
    public static RootResult Solve(double a, double b, double epsilon)
    {
        var startA = a;
        var startB = b;

        Console.WriteLine("1. МЕТОД ПОЛОВИННОГО ДЕЛЕНИЯ");
        Console.WriteLine("   Отрезок [" + NumberFormat.Format(a, 2) + "; "
            + NumberFormat.Format(b, 2) + "], точность eps = "
            + NumberFormat.Format(epsilon, 6));
        Console.WriteLine("   f(a) = " + NumberFormat.Format(Equation.Value(a), 4)
            + ", f(b) = " + NumberFormat.Format(Equation.Value(b), 4)
            + "; f(a)*f(b) < 0 — корень отделён.");
        Console.WriteLine();
        Console.WriteLine(string.Format("{0,4}{1,14}{2,14}{3,14}{4,15}{5,14}",
            "n", "a", "b", "c=(a+b)/2", "f(c)", "b−a"));

        var steps = 0;
        double c;
        do
        {
            c = 0.5 * (a + b);
            steps++;
            Console.WriteLine(string.Format("{0,4}{1,14}{2,14}{3,14}{4,15}{5,14}",
                steps, Format(a, 8), Format(b, 8), Format(c, 8),
                Format(Equation.Value(c), 8), Format(b − a, 8)));
            if (Equation.Value(a) * Equation.Value(c) <= 0)
            {
                b = c;
            }
            else
            {
                a = c;
            }
        } while (b − a >= epsilon);

        var root = 0.5 * (a + b);
        Console.WriteLine();
        Console.WriteLine("   Корень: x = " + NumberFormat.Format(root, 8)
            + ", f(x) = " + NumberFormat.Format(Equation.Value(root), 8)
            + ", шагов: " + steps);
        Console.WriteLine();

        return new RootResult("половинного деления",
            "a=" + NumberFormat.Format(startA, 1) + ", b=" + NumberFormat.
    Format(startB, 1),
```

```
            root, root, steps);
    }

    private static string Format(double value, int digits)
    {
        return NumberFormat.Format(value, digits);
    }
}
```

ChordSolver.cs

```
using System;

// Метод хорд метод( секущих с неподвижным концом). Неподвижен тот конец
// отрезка, для которого f(x)*f''(x) > 0; начальным приближением служит
// противоположный конец. Вторая производная на [a, b] знакопостоянна, что
// обеспечивает сходимость. Последовательность приближений монотонна и
// подходит к корню с одной стороны.
static class ChordSolver
{
    public static RootResult Solve(double a, double b, double epsilon)
    {
        var strongAtA = Equation.Value(a) * Equation.SecondDerivative(a) > 0;
        var fixedEnd = strongAtA ? a : b;             // неподвижный конец
        var x = strongAtA ? b : a;                    // начальное
   приближение
        var x0 = x;

        Console.WriteLine("2. МЕТОД ХОРД");
        Console.WriteLine("   Отрезок [" + NumberFormat.Format(a, 2) + "; "
            + NumberFormat.Format(b, 2) + "], точность eps = "
            + NumberFormat.Format(epsilon, 6));
        Console.WriteLine("   f''(x) = 6x − 0.2 > 0 на отрезке; f(x)*f''(x) >
    0 при x = "
            + NumberFormat.Format(fixedEnd, 2) + ", поэтому неподвижен конец
   "
            + (strongAtA ? "a" : "b") + ", x0 = " + NumberFormat.Format(x0,
   2) + ".");
        Console.WriteLine();
        Console.WriteLine(string.Format("{0,4}{1,14}{2,15}{3,14}{4,16}",
            "n", "x(n)", "f(x(n))", "x(n+1)", "|x(n+1)−x(n)|"));

        var steps = 0;
        double next;
        double delta;
        do
        {
            var fx = Equation.Value(x);
            next = x − fx / (fx − Equation.Value(fixedEnd)) * (x − fixedEnd);
            delta = Math.Abs(next − x);
            steps++;
            Console.WriteLine(string.Format("{0,4}{1,14}{2,15}{3,14}{4,16}",
                steps, Format(x, 8), Format(fx, 8), Format(next, 8), Format(
   delta, 8)));
            x = next;
        } while (delta >= epsilon && Math.Abs(Equation.Value(x)) >= epsilon);

        Console.WriteLine();
```

```
        Console.WriteLine("   Корень: x = " + NumberFormat.Format(x, 8)
            + ", f(x) = " + NumberFormat.Format(Equation.Value(x), 8)
            + ", шагов: " + steps);
        Console.WriteLine();

        return new RootResult("хорд",
            "a=" + NumberFormat.Format(a, 1) + ", b=" + NumberFormat.Format(b, 1)
            + ", x0=" + NumberFormat.Format(x0, 1),
            x0, x, steps);
    }

    private static string Format(double value, int digits)
    {
        return NumberFormat.Format(value, digits);
    }
}
```

NewtonSolver.cs

```
using System;

// Метод Ньютона метод( касательных). За начальное приближение берётся тот
// конец отрезка, для которого f(x)*f''(x) > 0 условие( сходимости Фурье).
// В точке x_n проводится касательная, её пересечение с осью Ox даёт x_{n+1}:
//     x_{n+1} = x_n - f(x_n) / f'(x_n).
// Метод сходится квадратично, приближение идёт с одной стороны.
static class NewtonSolver
{
    public static RootResult Solve(double a, double b, double epsilon)
    {
        var strongAtA = Equation.Value(a) * Equation.SecondDerivative(a) > 0;
        var x = strongAtA ? a : b;
        var x0 = x;

        Console.WriteLine("3. МЕТОД НЬЮТОНА КАСАТЕЛЬНЫХ()");
        Console.WriteLine("   Отрезок [" + NumberFormat.Format(a, 2) + "; "
            + NumberFormat.Format(b, 2) + "], точность eps = "
            + NumberFormat.Format(epsilon, 6));
        Console.WriteLine("   f(x)*f''(x) > 0 при x = " + NumberFormat.Format(x0, 2)
            + ", поэтому x0 = " + NumberFormat.Format(x0, 2) + ".");
        Console.WriteLine();
        Console.WriteLine(string.Format("{0,4}{1,14}{2,15}{3,14}{4,16}",
            "n", "x(n)", "f(x(n))", "x(n+1)", "|x(n+1)-x(n)|"));

        var steps = 0;
        double next;
        double delta;
        do
        {
            var fx = Equation.Value(x);
            next = x - fx / Equation.Derivative(x);
            delta = Math.Abs(next - x);
            steps++;
            Console.WriteLine(string.Format("{0,4}{1,14}{2,15}{3,14}{4,16}",
                steps, Format(x, 8), Format(fx, 8), Format(next, 8), Format(delta, 8)));
```

```
            x = next;
        } while (delta >= epsilon && Math.Abs(Equation.Value(x)) >= epsilon);

        Console.WriteLine();
        Console.WriteLine("   Корень: x = " + NumberFormat.Format(x, 8)
            + ", f(x) = " + NumberFormat.Format(Equation.Value(x), 8)
            + ", шагов: " + steps);
        Console.WriteLine();

        return new RootResult("касательных Ньютона()",
            "a=" + NumberFormat.Format(a, 1) + ", b=" + NumberFormat.Format(b, 1)
            + ", x0=" + NumberFormat.Format(x0, 1),
            x0, x, steps);
    }

    private static string Format(double value, int digits)
    {
        return NumberFormat.Format(value, digits);
    }
}
```

CombinedSolver.cs

```
using System;

// Комбинированный метод хорд и касательных. Корень уточняется одновременно
// с двух сторон: по недостатку – методом хорд, по избытку – методом
// касательных. Обозначим x – приближение по недостатку, y – по избытку.
// Неподвижный при хордах конец и конец для касательных выбираются по знаку
// f(x)*f''(x). На каждом шаге
//     x_{n+1} = x_n – f(x_n)/(f(x_n) – f(y_n)) * (x_n – y_n),
//     y_{n+1} = y_n – f(y_n)/f'(y_n).
// Процесс заканчивается, когда y_n – x_n < eps; корень лежит между x_n и y_n.
static class CombinedSolver
{
    public static RootResult Solve(double a, double b, double epsilon)
    {
        var strongAtA = Equation.Value(a) * Equation.SecondDerivative(a) > 0;
        // Метод хорд идёт от слабого конца, метод касательных – от сильного
        var x = strongAtA ? b : a;    // по недостатку хорды()
        var y = strongAtA ? a : b;    // по избытку касательные()
        var x0 = x;
        var y0 = y;

        Console.WriteLine("4. КОМБИНИРОВАННЫЙ МЕТОД ХОРД И КАСАТЕЛЬНЫХ");
        Console.WriteLine("   Отрезок [" + NumberFormat.Format(a, 2) + "; "
            + NumberFormat.Format(b, 2) + "], точность eps = "
            + NumberFormat.Format(epsilon, 6));
        Console.WriteLine("   f''(x) > 0; f(x)*f''(x) > 0 при x = "
            + NumberFormat.Format(y0, 2) + ", поэтому касательные строятся из x = "
            + NumberFormat.Format(y0, 2) + ", хорды – из x = " + NumberFormat.Format(x0, 2) + ".");
        Console.WriteLine();
        Console.WriteLine(string.Format("{0,4}{1,14}{2,14}{3,15}{4,15}{5,14}",
```

```
            "n", "x(n)", "y(n)", "f(x(n))", "f(y(n))", "y(n)-x(n)"));

        var steps = 0;
        do
        {
            var fx = Equation.Value(x);
            var fy = Equation.Value(y);
            var xNext = x - fx / (fx - fy) * (x - y);
            var yNext = y - fy / Equation.Derivative(y);
            steps++;
            Console.WriteLine(string.Format("{0,4}{1,14}{2,14}{3,15}{4,15}{5,14}",
                steps, Format(x, 8), Format(y, 8), Format(fx, 8), Format(fy, 8),
                Format(y - x, 8)));
            x = xNext;
            y = yNext;
        } while (y - x >= epsilon);

        var root = 0.5 * (x + y);
        Console.WriteLine();
        Console.WriteLine("   Корень заключён между " + NumberFormat.Format(x, 8)
            + " и " + NumberFormat.Format(y, 8));
        Console.WriteLine("   Корень: x = " + NumberFormat.Format(root, 8)
            + ", f(x) = " + NumberFormat.Format(Equation.Value(root), 8)
            + ", шагов: " + steps);
        Console.WriteLine();

        return new RootResult("комбинированный",
            "x0=" + NumberFormat.Format(x0, 1) + ", y0=" + NumberFormat.Format(y0, 1),
            x0, root, steps);
    }

    private static string Format(double value, int digits)
    {
        return NumberFormat.Format(value, digits);
    }
}
```

IterationSolver.cs

```
using System;

// Метод простых итераций. Уравнение f(x) = 0 приводится к виду x = phi(x),
// где phi(x) = x - f(x)/k. Параметр k выбирается одного знака с f'(x) и по
// модулю больше Q/2, где Q = max|f'(x)| на [a, b]; тогда |phi'(x)| <= q < 1
// и последовательность x_{n+1} = phi(x_n) сходится к корню. Оценка точности:
//     |xi - x_n| <= q/(1-q) * |x_n - x_{n-1}|.
static class IterationSolver
{
    public static RootResult Solve(double a, double b, double epsilon)
    {
        var q0 = MaxDerivative(a, b);
        var k = Equation.Derivative(0.5 * (a + b));
        if (Math.Abs(k) <= 0.5 * q0)
        {
```

```
            k = Math.Sign(Equation.Derivative(a)) * q0;
        }
        var q = MaxPhiDerivative(a, b, k);

        var x = 0.5 * (a + b);
        var x0 = x;

        Console.WriteLine("5. МЕТОД ИТЕРАЦИЙ");
        Console.WriteLine("   Отрезок [" + NumberFormat.Format(a, 2) + "; "
            + NumberFormat.Format(b, 2) + "], точность eps = "
            + NumberFormat.Format(epsilon, 6));
        Console.WriteLine("   max|f'| = " + NumberFormat.Format(q0, 4)
            + "; принято k = " + NumberFormat.Format(k, 4)
            + " знак( k совпадает со знаком f', |k| > max|f'|/2).");
        Console.WriteLine("   phi(x) = x - f(x)/k; |phi'(x)| <= q = "
            + NumberFormat.Format(q, 4) + " < 1 - условие сходимости
выполнено.");
        Console.WriteLine("   Начальное приближение x0 = " + NumberFormat.
Format(x0, 2) + ".");
        Console.WriteLine();
        Console.WriteLine(string.Format("{0,4}{1,14}{2,15}{3,16}",
            "n", "x(n)", "f(x(n))", "|x(n)-x(n-1)|"));

        var steps = 0;
        var delta = double.PositiveInfinity;
        do
        {
            var fx = Equation.Value(x);
            var next = x - fx / k;
            delta = Math.Abs(next - x);
            steps++;
            Console.WriteLine(string.Format("{0,4}{1,14}{2,15}{3,16}",
                steps, Format(x, 8), Format(fx, 8), Format(delta, 8)));
            x = next;
        } while (delta >= epsilon);

        var error = q / (1 - q) * delta;
        Console.WriteLine();
        Console.WriteLine("   Корень: x = " + NumberFormat.Format(x, 8)
            + ", f(x) = " + NumberFormat.Format(Equation.Value(x), 8)
            + ", шагов: " + steps);
        Console.WriteLine("   Апостериорная оценка погрешности: |xi - x| <= q
/(1-q)*|dx| = "
            + NumberFormat.Format(error, 8));
        Console.WriteLine();

        return new RootResult("итераций",
            "a=" + NumberFormat.Format(a, 1) + ", b=" + NumberFormat.Format(b
, 1)
            + ", x0=" + NumberFormat.Format(x0, 2),
            x0, x, steps);
    }

    // max|f'(x)| на отрезке перебор( с мелким шагом)
    private static double MaxDerivative(double a, double b)
    {
        var result = 0.0;
        const int samples = 1000;
        for (var i = 0; i <= samples; i++)
        {
```

```
            var x = a + (b - a) * i / samples;
            result = Math.Max(result, Math.Abs(Equation.Derivative(x)));
        }
        return result;
    }

    // max|phi'(x)| = max|1 - f'(x)/k| на отрезке
    private static double MaxPhiDerivative(double a, double b, double k)
    {
        var result = 0.0;
        const int samples = 1000;
        for (var i = 0; i <= samples; i++)
        {
            var x = a + (b - a) * i / samples;
            result = Math.Max(result, Math.Abs(1 - Equation.Derivative(x) / k
    ));
        }
        return result;
    }

    private static string Format(double value, int digits)
    {
        return NumberFormat.Format(value, digits);
    }
}
```

RootResult.cs

```
// Результат работы одного метода уточнения корня.
// Используется для построения сводной таблицы в конце программы.
class RootResult
{
    public RootResult(string method, string bounds, double x0, double root,
    int steps)
    {
        Method = method;
        Bounds = bounds;
        X0 = x0;
        Root = root;
        Steps = steps;
    }

    // Название метода
    public string Method { get; }

    // Задание отрезка и начального приближения в виде строки
    public string Bounds { get; }

    // Начальное приближение
    public double X0 { get; }

    // Найденный корень
    public double Root { get; }

    // Количество выполненных шагов
    public int Steps { get; }

    // Невязка: значение функции в найденном корне проверка( подстановкой)
```

```
    public double Residual => Equation.Value(Root);
}
```

NumberFormat.cs

```
using System;
using System.Globalization;

// Форматирование чисел для вывода в консоль
static class NumberFormat
{
    private const int CellWidth = 12;

    public static string Format(double value, int digits)
    {
        return value.ToString("F" + digits, CultureInfo.InvariantCulture);
    }

    // Число, выровненное по правому краю в поле фиксированной ширины
    public static string Cell(double value, int digits)
    {
        return Format(value, digits).PadLeft(CellWidth);
    }
}
```